

Aether — Backup & Disaster Recovery

Protect Every Workload Across Every Runtime

Atomic state persistence, pre-deploy snapshots, merge-or-replace restore strategies, and scheduled cleanup — built into the control plane.

Atomic Writes • 0o600 Permissions • Symlink Protection

Backup System Overview

Aether persists all workload state as JSON with atomic writes. Every backup captures metadata, workload definitions, and runtime state in a single portable file.

Core Commands

```
# Create a named backup of all workloads
$ aether backup --name pre-migration --description "Before K8s migration"

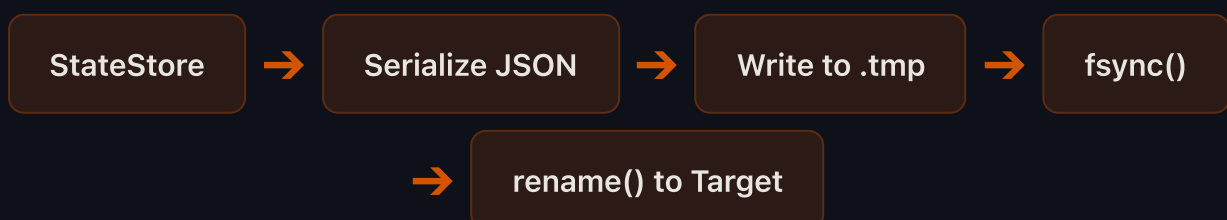
Backup saved: ~/.aether/backups/pre-migration.json
  Workloads: 12
  Version:   1.0
  Created:   2026-04-12T10:30:00Z

# Restore from a backup (full replace)
$ aether restore --from ~/.aether/backups/pre-migration.json

Restored 12 workloads from backup
```

Backup Metadata

Every backup includes a structured metadata header: version identifier, creation timestamp (RFC 3339), workload count, optional human-readable description, and the Aether version that created it. This metadata enables compatibility checks during restore and provides a clear audit trail of when and why each backup was taken.





Atomic writes guarantee the backup file is always in a consistent state — even if the process crashes mid-write, the original file remains intact.

Snapshot Manager

Pre-deploy and pre-migrate snapshots capture individual workload state, enabling surgical rollbacks without touching other workloads.



Pre-Deploy Snapshots

Before every deploy or migration, Aether automatically creates a snapshot of the affected workload. Each snapshot is named with the workload name and nanosecond-precision timestamp to guarantee uniqueness.



Rollback from Snapshot

If a deployment fails or introduces issues, roll back to the latest snapshot with a single command. Aether finds the most recent snapshot by workload name and restores it instantly.



Snapshot History

List all snapshots for a workload sorted chronologically. Each snapshot file contains the full backup structure including metadata, making it self-describing and portable.



Isolated Storage

Snapshots are stored in `~/.aether/snapshots/` with `0o600` permissions, separate from full backups. This isolation prevents accidental overwrites and simplifies retention policies.

Snapshot CLI

```
# Create a snapshot before manual changes
$ aether snapshot create my-app
```

```
Snapshot saved: ~/.aether/snapshots/my-app-20260412T103000-123456789.json
```

```
# List all snapshots for a workload
```

```
$ aether snapshot list my-app
```

```
my-app-20260410T090000-111111111.json
```

```
my-app-20260411T140000-222222222.json
```

```
my-app-20260412T103000-123456789.json (latest)
```

```
# Rollback to the latest snapshot
```

```
$ aether snapshot rollback my-app
```

```
Restored my-app from snapshot: my-app-20260412T103000-123456789.json
```

Backup Security

Every backup and snapshot file is protected by restrictive file permissions, symlink rejection, and strict name validation to prevent path traversal attacks.



0o600 Permissions

All backup and snapshot files are created with owner-only read/write permissions. No group or world access to sensitive workload state data.



Symlink Protection

When listing, reading, or deleting backups, Aether explicitly rejects symlinks. This prevents an attacker from tricking Aether into reading or overwriting files outside the backup directory.



Name Validation

Backup names are validated to reject path separators (/, \), traversal sequences (..), and null bytes. This blocks directory traversal attacks via crafted backup names.

Security Checks in Action

```
# Path traversal attempt - blocked
$ aether backup --name "../../../etc/passwd"
```

```
Error: Invalid backup name '../../../etc/passwd':
must not contain path separators or traversal sequences
```

```
# Symlink in backup directory - skipped
$ aether backup list
```

```
WARNING: Skipping symlink in backup directory: malicious-link.json
pre-migration.json      12 workloads  2026-04-10
daily-backup.json       8 workloads  2026-04-11
```

```
# Verify file permissions
$ ls -la ~/.aether/backups/
-rw----- 1 user user 4096 Apr 12 10:30 pre-migration.json
-rw----- 1 user user 3200 Apr 11 22:00 daily-backup.json
```

Protection	Threat	Mechanism
0o600 permissions	Unauthorized read access	POSIX file mode set after write
Symlink rejection	Path traversal via symlinks	<code>is_symlink()</code> check before read/delete
Name validation	Directory traversal via filenames	Reject <code>/</code> , <code>\</code> , <code>..</code> , <code>\0</code>
Atomic writes	Corrupted backups from crashes	Write to <code>.tmp</code> then <code>rename()</code>

Merge vs Replace Restore

Aether offers two restore strategies for different disaster recovery scenarios: full replacement and non-destructive merge.



Replace Restore

Creates a fresh StateStore and overwrites the current state entirely with the backup contents. Use this for full disaster recovery when the current state is corrupted or lost.



Merge Restore

Loads the existing state and adds only workloads from the backup that do not already exist. Existing workloads are never overwritten, making this safe for partial recovery scenarios.

Replace Strategy

```
# Full restore - replaces all current state
$ aether restore --from backup.json --strategy replace
```

```
Restored 12 workloads from backup
WARNING: Previous state has been fully replaced
```

Merge Strategy

```
# Merge restore - only adds missing workloads
$ aether restore --from backup.json --strategy merge
```

```
Merged 4 new workloads from backup (total in backup: 12)
Skipped 8 workloads already present in current state
```

Scheduled Backup with Cleanup

Keep Last N Backups

The BackupManager supports automatic cleanup of old backups. Call `cleanup_old_backups(keep)` to retain only the most recent N backups sorted by modification time. Older backups are deleted safely with symlink checks. This prevents unbounded disk growth while maintaining a rolling recovery window.

```
# Create a scheduled backup and clean up old ones
```

```
$ aether backup --name "daily-$(date +%Y%m%d)"
```

```
$ aether backup cleanup --keep 7
```

```
Cleaned up 3 old backups (kept 7 most recent)
```

```
# Automate with cron (daily at midnight, keep 7 days)
```

```
0 0 * * * aether backup && aether backup cleanup --keep 7
```

The merge strategy is ideal for recovering specific workloads from backup without disrupting workloads that are already running and healthy.

Protect Your Infrastructure

Aether's backup and disaster recovery system is built for production from day one.

0

Data Loss Events

2

Restore Strategies

ns

Snapshot Precision

Never Lose a Workload Again

Atomic writes, pre-deploy snapshots, symlink-safe backups, and automated cleanup — disaster recovery that works when you need it most.

```
# Quick start: backup, snapshot, and restore
$ aether backup --name before-release
$ aether snapshot create my-app
$ aether deploy my-app --spec new-version.yaml
$ aether snapshot rollback my-app    # if something goes wrong
$ aether backup cleanup --keep 7    # housekeeping
```

Aether — Universal Runtime Control Plane